Pure Rho and Cost: One Authored Calculus, Named Extensions, and Corrected Hypotheses

Mettapedia

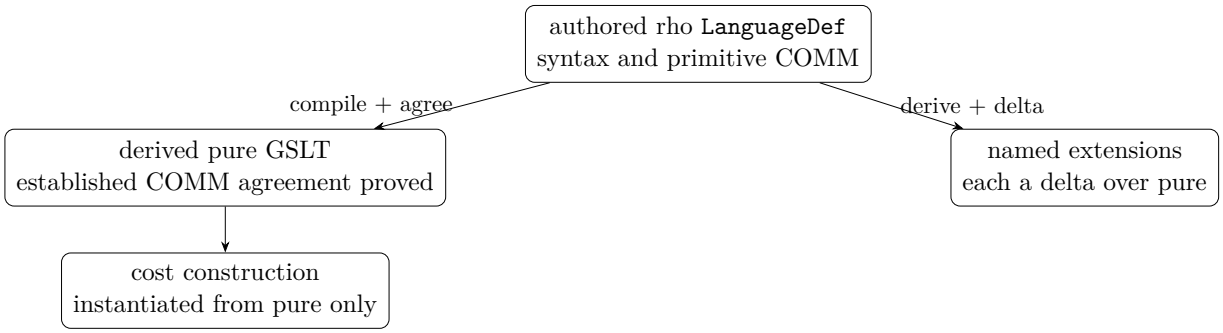
July 2026 (working paper)

Abstract

The rho-calculus is a process calculus in which every name is a quoted process. This paper fixes the exact calculus we formalize and implement: a *pure core* whose only operational reduction is communication, with every engine convenience recorded as a named extension derived from that core, never mixed into it. On this base we state the cost construction — wrapping terms and gating reductions by funding — together with two corrections to its published hypotheses: the injectivity assumption behind *quote-faithfulness* cannot hold for ordinary monad multiplication on any nontrivial signature monoid: joint injectivity of multiplication together with a two-sided unit forces the carrier to be a subsingleton. The repair is not a stronger monoid hypothesis. Funded execution instead forms an Atkey-style parameterized monad indexed by complete pre- and post-resource states. Exact causal receipts are retained for authority decisions, while prices and budget totals factor through explicit, generally non-injective aggregation maps. The obstruction, the parameterized monad laws, this authority/observation factorization, and a computable canonical section for pure structural congruence are formalized. Beyond the single calculus, Cost is developed as a construction on *continued interactive GSLTs* — languages equipped with one ordered interaction cut, a computable canonical section, and a position-sensitive continuation-retyping plan — and the generated funded language is itself such an object, with the object-closure laws of the resulting endofunctor programme largely discharged. A second compiled counterexample is recorded: the global raw fiber-stability hypothesis of our earlier normalization architecture is false for rho, and its repair moves the entire equational theory onto typed open carriers with proof-relevant elaboration, on which exact canonical laws cut out the full subcategory hosting strict Cost_1. Public raw admission checks both the cost grammar and whole-object binder scope, including purse locations. On that admitted fragment, the budgeted Lean executor refines a declarative funded trace and erases to a path of the pure-rho GSLT mechanically derived from the authored language definition, with exactly one COMM step per funded event. The CeTTa runtime realization of the funded semantics — located purses, causal receipts, honest budgets, and occurrence-claiming parallel waves — is presented with runnable, regression-tested showcases. Claims are marked as formalized or open; nothing here is asserted beyond its marker.

1 Introduction

A calculus and its implementation drift apart unless the boundary between them is a maintained artifact. Reference implementations of the rho-calculus add conveniences — direct execution of quoted processes, polyadic inputs, name extrusion — that are individually reasonable and collectively fatal to the question "what exactly has been proved?" Our answer is architectural:



The authored `LanguageDef` is the intended source presentation for the strict syntax and primitive COMM rule. The present Lean development proves that its compiled matcher, normalization, substitution, and supported COMM contracta agree with the independently established semantic development. The pure GSLT used below is itself derived from that authored declaration, so the agreement theorem does not create a second operational authority. Named extensions import the pure theory and add explicit deltas; the pure theory and the runtime cost profile import none of them. Full equivalence between the complete derived contextual relation and the independently established relation is a separate theorem obligation, not assumed here.

Rho, for us, is not a formalization exercise. It is a system with three living parts — a running concurrent engine, a mathematical theory with corrected hypotheses, and a machine-checked formalization tying the two together — and the paper is ordered accordingly: the calculus (Section 2) and the engine running it (Section 4) come first, so that the mathematics of funding (Section 6 onward) and the formal agreement theorems (Sections 11–12) arrive after the phenomena they explain.

2 The pure calculus

Syntax. Processes P and names x are mutually defined:

$$P \ ::= \ 0 \ \mid \ \mathsf{for}(y \leftarrow x)\, P \ \mid \ x!(P) \ \mid \ P \,|\, P \ \mid \ *x \qquad\qquad x \ ::= \ @P.$$

A name is a quoted process $@P$; the process $*x$ is the *drop* of a name. Quotation and dropping are the calculus's reflection: processes can be passed as data and data can be run — but, as we now make precise, only through communication.

Two equivalences, kept separate. The calculus has two distinct static equivalences, and sorting laws into the correct one is load-bearing:

- *Name equivalence* contains the cancellation $@(*x) \equiv_N x$: quoting the drop of a name gives back the name. This is an equation between *names*.
- *Process structural congruence* contains alpha-equivalence and the commutative-monoid laws for parallel composition ($P|Q \equiv Q|P$, $(P|Q)|R \equiv P|(Q|R)$, $P|0 \equiv P$).

Neither equivalence contains a rule that *runs* anything. In particular, $*(@P) \to P$ is *not* a law of the pure calculus, neither as an equation nor as a reduction.

One reduction. The only operational rule is communication:

$$\mathsf{for}(y \leftarrow x)\, P \mid x!(Q) \;\longrightarrow\; P\{@Q/y\},$$

where the substitution is semantic: when $P\{@Q/y\}$ replaces y inside a drop, $*y$ becomes Q. Dequotation therefore happens *only* through substitution after a communication — you can run exactly the code you were sent.

A positive and a negative example.

$$\underbrace{\mathsf{for}(y \leftarrow @0)\,(*y) \mid @0!(x!(0))}_{\text{reducible}} \;\longrightarrow\; x!(0) \qquad\qquad \underbrace{*(@\,x!(0))}_{\text{stuck in pure rho}} \;\not\longrightarrow$$

The left process communicates on the name @0 and the received quoted process runs via substitution. The right process is a free-standing drop of a quotation; in the pure calculus it is inert. [**formalized**] for the concrete one-step relation (communication is its only constructor); the corresponding negative example is part of the extension test suite described in Section 5.

3 Running quoted code without extending the calculus

The convenience that tempts implementations to add a primitive $*(@P) \to P$ step is already expressible, at the cost of one honest communication, by a two-process *runner* idiom:

$$\mathit{run}!(@P) \mid \mathsf{for}(y \leftarrow \mathit{run})\,(*y) \;\longrightarrow\; P.$$

The drop fires through substitution on a received name — pure rho — and the execution of a quoted process costs exactly one communication event, which the cost semantics of Section 6 meters honestly. Nothing about expressivity forces an execution primitive into the calculus; adding one changes the step relation, the stuck states, and every cost ledger, in exchange for saving a single COMM.

4 The running engine: funded rho in CeTTa

So much for the rules on paper — now press enter. Rho is not a formalization exercise: it is a running concurrent engine, and this section shows it running before any more theory is stated. The CeTTa runtime (`src/rhocalc_core.c`) implements exactly the calculus of Section 2 under `-lang rhocalc` — six constructors, COMM the only reduction, the free drop honestly inert — and, under `-profile cost`, a funded semantics whose mathematical laws are the subject of Section 6 onward. The showcase programs below ship with the engine in `examples/rho/` and run as regression tests; every output shown is the engine's actual output.

4.1 A vending machine that cannot be talked into anything

Watch a funded reaction refuse to fire. In the concrete syntax, `{P}s` is a process signed by `s`, and `purse c {s : ()}` is a store on channel `c` holding one token signed `s`. A sale is a handshake between a machine receive and a customer send, and the rule is *located*: the sale consumes one token per endpoint signature, drawn only from purses sitting on the `vend` channel itself. No ambient bank account exists anywhere.

```
{for ($order <- vend) {{0}served}}machine
| {for ($order <- vend) {{0}served}}machine
| {vend!({0}coin)}alice
| {vend!({0}coin)}bob
| purse vend {machine : machine : ()}
| purse vend {alice : ()}
```

The till holds two `machine` tokens; alice has a coin in the channel purse; bob has none. Run it:

```
purse vend {()} | purse vend {machine : ()} | {0}served
| {for ($order <- vend) {{0}served}}machine | {vend!({0}coin)}bob
```

Now look at what the machine did *not* do. Bob's order and a willing machine endpoint stand facing each other, forever. The reaction is enabled in the pure calculus; in the cost profile it is not slowed, not queued, not retried — it is *disabled*, because bob's share of the demand has no cover. That is the first thing to internalize about funded rho: cost is not a meter bolted onto the side of an evaluator. An unfunded reaction cannot fire, in the same way an ill-typed program cannot compile. And because names are quoted processes, a purse's location follows name equivalence, not spelling: a purse on `@{*pay}` funds reactions on `pay` — aliases of a channel are the same account.

4.2 Receipts: the run tells you its own causal history

What does a run return? Not a number. Here is a three-stage supply chain — farm ships, roaster roasts, café brews — on one channel with one purse holding one token per stage, run through the accounted trace library (`lts:rho:cost`):

```
(lts:rho:cost:receipt
  ((lts:rho:cost:event 0 () ((lts:rho:cost:funding market farm)) farm)
   (lts:rho:cost:event 1 (0 0) ((lts:rho:cost:funding market roaster)) roaster)
   (lts:rho:cost:event 2 (1 1) ((lts:rho:cost:funding market cafe)) cafe))
  (rho:cost:par (rho:cost:purse market rho:cost:stack-empty)
                (rho:cost:signed rho:nil espresso)))
```

Each event carries a fresh identifier, a *cause list*, one funding record per token spent, and the consumed signature. Read event 1's cause list: `(0 0)` — zero, twice. Why twice? Because event 1 consumed two distinct occurrences that event 0 produced — the roaster's continuation *and* the purse tail left after the farm's spend — and the receipt keeps one arc per consumed occurrence, repetitions included. Two independent settlements on separate channels get empty cause lists even though one event number is emitted first: emission order is bookkeeping, causality is consumption. A receipt is a located, measured pomset of spend events; the printed sequence is one linearization of it. And deliberately, nowhere in it is "the cost": totals, prices, and critical-path depths are monoid homomorphisms computed *from* the receipt afterward, in exactly the authority-then-observation order that Section 6 below makes into a theorem.

4.3 Budgets that never lie

The exhaustive enumerations never truncate silently — and that is a promise with teeth, because funding search is genuine exact cover: a demand of ten unit coins over twenty unit purses admits $\binom{20}{10} = 184{,}756$ distinct covers, and the unlimited frontier will enumerate every one if asked. Bounded execution is strictly opt-in, takes *two* separate allowances — reaction fuel and cover-search work — and returns a status that never lies: `quiescent` only after exhaustive search *proves*

no funded successor exists; `fuel-exhausted` when the firing allowance ends first (at fuel zero the machine does not even search); `search-exhausted` when cover search stops while further search remains semantically relevant. An out-of-fuel machine says "I ran out." It never says "there was nothing there." The three verdicts, and a two-cover spend showing why search deserves its own meter, are runnable in `budget_meter.metta`.

4.4 Parallel waves, and receipts as a free observer

With `-num-threads N`, compatible firings execute in OS-thread waves. Before a worker fires, it atomically claims the *exact occurrences* its plan touches — both endpoints and every funding token — in a fixed sorted order; a wave admits only pairwise-disjoint claims, and each committed wave is re-validated against occurrence accounting. The sequential semantics stays the reference: a threaded run is a legal parallel refinement, not a new relation, and because residuals are kept canonical, the eight-settlement showcase (`parallel_settlement.mrho`) produces byte-identical output sequentially and at every tested thread count. Receipts are an *optional observer* on this path: a state-only run and a receipt-observed run consume identical resources and reach the same residual, checked by a scripted transparency gate.

4.5 How much of this is proved?

The engine and the formalization meet at a differential boundary, and we keep its status exact. On the Lean side, the budgeted executor's runs refine declarative funded traces and erase to derived pure-GSLT paths (Section 12 below), with the axiom-audited theorems of this paper. Across the boundary: 146 sequential/threaded prefix cases agree; 308 compiled-C receipts replay as Lean derivations while tampered receipts are rejected; the full runtime suite passes under thread and address/undefined-behaviour sanitizers; and a commit-audit build re-derives every committed parallel firing inside the wave commit. This is bounded adversarial evidence about the compiled engine — deliberately never presented as a universal theorem about C.

5 Extensions as named deltas

Engine conveniences are legitimate — as *named* extensions. The discipline has three parts.

Structure. Each extension is a delta over the pure theory: the extension module imports the pure calculus and adds constructors, equations, or rules; the pure modules import no extension. Distinct conveniences are distinct deltas — at minimum, a minimal *execution* extension (pure rho plus $*(@P) \to P$) is kept separate from a *reference-implementation profile* (polyadic input, replicated input, name extrusion, native data operations), because they change different parts of the theory and generate different cost events. In the formal library these deltas may be collected in one clearly labelled extended-rho namespace; the runtime's cost profile instantiates none of them.

Theorems, not conventions. Each delta carries a positive and a negative theorem: the free drop *cannot* step in pure rho [**formalized**]; the free drop *does* step in the execution extension [**formalized**]; and the cost profile is instantiated from the pure calculus specifically. The two reduction claims are proved for the declarative language definitions. The runtime import/profile boundary remains a separate closure gate. A text search for stray constructors is useful supplementary evidence; the load-bearing boundary is the import graph and these theorems.

The delta as a document. A reference implementation's distance from the paper calculus is itself worth recording. The audit of the implementation we compare against (the `mettail` engine's rho language) yields exactly such a delta: an execution rewrite, polyadic and persistent communication, extrusion, and native data — all useful, none of them part of the calculus whose metatheory we prove. Recording the delta turns "the paper and the code disagree" from folklore into data.

6 The cost construction: corrected hypotheses

The cost construction wraps terms and gates reduction by funding: a wrapped redex fires only when the wrapper supplies a token drawn from a commutative monoid $(S, *, e)$ of *signatures*, and each reduction consumes its funding and emits a receipt. The running engine of Section 4 realizes this semantics; here we state the mathematical hypotheses, two of which require correction.

6.1 The quote-faithfulness obstruction

Call a map *quote-faithful* when it is injective on the signature keys reachable in a given theory. The published development uses per-argument injectivity of the signature operation. Three separate claims hide under this phrase, with three different fates.

(a) Fixed-element level. "Multiplication by a fixed s is injective" ($s * a = s * b \Rightarrow a = b$) is exactly *cancellativity* of the monoid — and it *fails* in general:

$$\max(3,1) = \max(3,2) \qquad\qquad \infty + 1 = \infty + 2.$$

Idempotent (tropical) aggregation and saturating budgets are therefore outside the hypothesis. Free signatures — finite multisets of generators — are cancellative, so the intended examples survive *at this fixed-element level once the hypothesis is stated.* [**formalized**]

(b) Multiplication level. The monad multiplication $\mu : \mathrm{Cost}^2 \to \mathrm{Cost}$ flattens two layers of wrapping, combining an outer and an inner signature into one key. Cancellativity does *not* make this flattening injective, because joint injectivity of $(a, b) \mapsto a * b$ is a unique-factorization property, not a cancellation property:

$$1 + 2 \;=\; 0 + 3 \quad \text{in } (\mathbb{N}, +), \qquad\qquad \{a\} \uplus \{b, c\} \;=\; \{a, b\} \uplus \{c\} \quad \text{for free multisets.}$$

Both monoids are cancellative; in both, distinct outer/inner factorizations flatten to the same key. More generally, if a binary operation with a two-sided unit were jointly injective, then $(e, a) = (a, e)$ for every a, forcing every element to equal e. Thus:

$$\mathrm{inj}\big((a, b) \mapsto a * b\big) \quad \Longleftrightarrow \quad S \text{ is a subsingleton.}$$

No nontrivial monoid—including words, free multisets, or natural-number costs—can satisfy the published multiplication-level requirement. This is an impossibility theorem, not a missing hypothesis. [**formalized**]

The same obstruction is forced directly by the monad unit laws: $\mu \circ \eta_{TX} = \mathrm{id}_{TX} = \mu \circ T\eta_X$. For a nontrivial grade, the outer-unit and inner-unit placements are distinct factorizations with the same flattened image. Thus changing the signature carrier cannot restore injectivity of ordinary monad multiplication.

(c) Unit level. The unit η must not fabricate funding. In the corrected semantics, unit is the empty execution at a fixed resource state and carries the empty receipt. Every nonempty funded execution advances the event counter; hence no nonempty execution can masquerade as an identity. The ordinary "freely available unit token" argument is withdrawn. **[formalized]** for the concrete funded semantics.

6.2 The typed repair: pre/post indexed execution

Let I be the type of complete funded states: event counter, live resource components, provenance, and the proofs required to continue safely. For $i, j \in I$, let $T(i, j, X)$ be the type of funded executions from i to j returning a value in X. The operations are

$$\mathrm{pure}_i : X \to T(i, i, X), \qquad \mathrm{bind} : T(i, j, X) \times (X \to T(j, k, Y)) \to T(i, k, Y).$$

The shared middle index is part of the type. An execution ending at j cannot be composed with one beginning at a different resource state; invalid funding composition is therefore untypeable. Empty execution supplies pure, path concatenation supplies bind, and the two unit laws and associativity are proved from the corresponding path laws. **[formalized]**

To feel what the index buys, watch a purse with two coins. Run A spends one: its type is $T(2, 1, \_)$. Run B spends one more: $T(1, 0, \_)$. They compose, and the composite says $T(2, 0, \_)$ — two coins in, none out. Now try to follow A with a run that spends *two* coins. That run's type begins at 2; but after A the world is at 1, and bind demands the middle indices meet. There is no term to write. The overdraft is not caught at run time — it is *unsayable*, the way "three meters plus four seconds" is unsayable in dimensional analysis. And the unit is honest for the same reason: pure has type $T(i, i, X)$, an execution that provably spends nothing, where the withdrawn argument needed a unit token available for free. What the impossibility theorem says a single flattened object must forget, the pair of indices simply refuses to forget.

This is an Atkey-style parameterized monad [5]. It is also the pre/post-state specialization of the broader graded and parametric-effect perspective [6, 7]: a monoid grade summarizes effects, while a category grade records which state transitions may compose. The present formalization constructs the resource transition category and the parameterized monad instance; it does not claim a fully general category-graded monad for arbitrary result families.

6.3 Authority is exact; pricing is an observation

A causal receipt retains event occurrences, funding records, and their order. Sequential composition concatenates these exact receipts. Budget totals, additive prices, multiplicative prices, and signature multisets are obtained only afterward by monoid homomorphisms:

$$\mathrm{Receipt} \longrightarrow \mathrm{Aggregate} \longrightarrow \mathrm{Price}.$$

The first object is suitable for replay and authority decisions; the later objects intentionally forget information. In particular, the raw account functor is proved to factor through the exact authority-receipt functor and an explicit aggregation homomorphism. The aggregation is not injective, and no total reconstruction from it exists. **[formalized]**

6.4 What stands

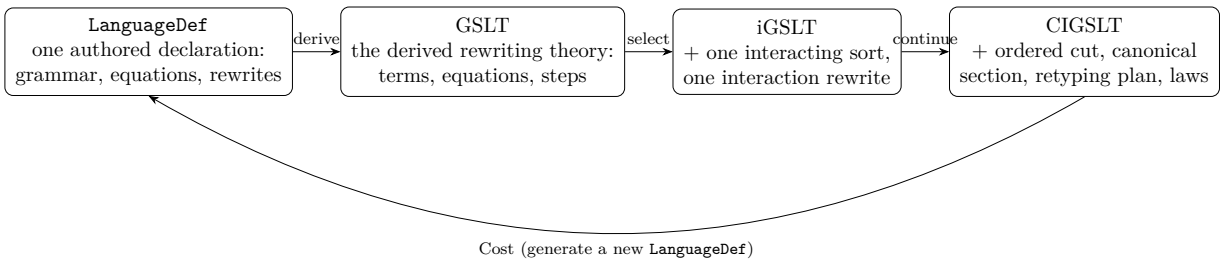
The concrete runtime already follows the corrected shape: it retains receipts, gates every firing by actual funding, and derives totals by folding. Its receipt replay, sequential/threaded agreement, and

sanitizer evidence do not depend on joint injectivity of signature multiplication. What is withdrawn is the ordinary-monad faithfulness claim. What replaces it is stronger where authority matters: exact indexed transitions and exact receipts, with every lossy view named as an observation.

7 The continued category: Cost as a construction on languages

The corrected hypotheses of Section 6 are stated for one calculus. The deeper architectural claim — Meredith's "wrapping by construction" — is that Cost is a *construction on languages*: it should consume a language equipped with one interaction and return another such language, so that wrapping composes.

Here is the question in plain terms. You wrapped rho once and got a new language: new constructors (signed terms, purses, token stacks), new equations, exactly one new rule. Can you wrap *that*? If Cost is a real construction and not a one-off trick, the answer must be yes — mechanically, with no human editing the generated language first. But "wrap it again" only makes sense if the generated language arrives carrying the same *kind* of equipment the original did: which constructor is the handshake, where the continuations sit, how to canonicalize. So the right object is not a language; it is a language *with sockets* — and Cost is the factory that consumes a socketed language and manufactures another socketed language, sockets included. Making that precise forced a new compositional object, and we consider the result one of the genuinely novel architectures of this development. This section states it exactly as formalized. The staging is a strict tower — each stage adds one kind of structure and forgets none:



The loop closes at the left: Cost consumes a continued object and emits a new *authored-style declaration* — the generated funded language of Section 7.3 — which re-enters the same tower. Nothing in the tower is hand-written twice; every stage is derived or selected from the stage before it.

7.1 Continued interactive GSLTs

A *generated syntactic labelled transition system* (GSLT) is the rewriting theory mechanically derived from one authored `LanguageDef`; an *interactive* GSLT (iGSLT) additionally fixes a validated presentation with one distinguished interacting sort and one interaction rewrite. A *continued interactive GSLT* (`CIGSLT`) equips an iGSLT with exactly the data Cost needs to fire again:

- one ordered authored *interaction cut*: the core contact constructor, and a program operand and an environment operand, each an authored constructor together with its selected *continuation position* and continuation schema variable;
- a *computable contextual open section*: canonicalization on every declaration-derived open sorted fiber, retaining sort, free-variable context, binder context, and reflective scope (Section 10);

- a *continuation retyping plan*: the finite declaration-level recipe that sends every constructor except the two selected principals to a hereditarily *wrapped* copy, and the two principals to *cut-aware base* copies whose continuation parameter alone is retyped into the wrapped fiber;
- closure laws, each a proposition over the plan: every equation and every reflective presentation of the source survives the retyping with sorted base and wrapped images (`EquationsRetypable`, `ReflectivePresentationsRetypable`); the selected redex and contractum remain sorted after their continuation positions move to the wrapped fiber (`RedexRetypable`, `Wrappable`); the authored envelope around the interaction core is a signature context stable under retyping (`sourceEnvelopeStable`); every bare-collection constructor — whose label is invisible in raw syntax — is explicitly non-principal (`bareCollectionConstructorsWrapped`); and canonicalization preserves the non-principal typed constructor fragment, so normalization can never manufacture an interaction principal.

The last two laws deserve emphasis because each answers a concrete soundness hazard. A bare collection hides its constructor label in the raw pattern, so a typing derivation chooses its rule non-syntactically; requiring all such rules to be non-principal makes every hidden choice harmless *independently of which candidate a derivation picked*. And requiring the canonicalizer to preserve the non-principal fragment closes the loop between normalization and interaction: rearranging authored static structure is allowed, manufacturing a redex is not.

In Lean the object is one structure, reproduced here abridged (`ContinuedCategory.lean`):

```
structure CIGSLT where
  theory : IGSLT
  cut : InteractionCutPresentation theory
  openCanonical : ComputableContextualOpenSection theory
  continuationRetyping : ContinuationRetypingPlan cut
  bareCollectionConstructorsWrapped : ... -- hidden labels are non-principal
  openCanonicalPreservesWrappedConstructorTyping : ...
  equationsRetypable : EquationsRetypable continuationRetyping
  reflectivePresentationsRetypable : ...
  sourceEnvelopeStable : ContinuationStableContext cut ...
  redexRetypable : continuationRetyping.RedexRetypable
  wrappable : continuationRetyping.Wrappable
```

Every law is a proposition over `(theory, cut, plan)` alone — a deliberate design point: the closure obligations of the *generated* object are statable before that object is fully assembled, which is what Section 9 exploits.

7.2 Position-sensitive retyping is not a morphism

The retyping plan is deliberately *not* a uniform signature morphism, and this is a theorem-shaped fact rather than an implementation convenience. The generated language declares base copies of every source constructor and wrapped copies of the non-principals; the two principals receive *cut-aware* base copies whose continuation parameter is wrapped while every other parameter is base. Consequently no single symbol action covers the retyping: occurrences of the wrapped sort must map differently at continuation positions than elsewhere. The formal development therefore transports typing derivations along the plan with a dedicated induction (`mapCostStaticGenerated`), stated over exactly (theory, cut, plan) plus the bare-collection law — and over nothing else. This minimal signature is what makes the construction iterate without circularity: the generated object inhabits it before the generated object's own closure laws are discharged. [**formalized**]

7.3 The generated funded language

From a `CIGSLT` the construction produces the complete funded language in two stages. The *continuation signature* is the generated base/wrapped grammar above. The *whole language* adds a small administrative apparatus — a contact constructor, a signing wrapper, funding and token-stack constructors — and exactly one rewrite, the *whole-redex rule*: the base-mapped source redex, signed and placed beside a funding stack whose head carries the demanded signature, contracts in one step to the wrapped contractum beside the unconsumed stack tail. The generated source pattern is positively characterized — it is literally one explicit contact of a signed redex with a matching funded stack (`CostInteraction.lean`, theorem `costWholeRedexSource_eq`):

```
-- left side of the one generated rewrite
contact( signed( <base-mapped source redex> , $sig ),
        funding( stackCons($sig, $tail) ) )
-- right side
contact( <wrapped source contractum> , funding($tail) )
```

The rule consumes the stack head and carries the tail variable through unchanged, and no rule creates a replacement token. [**formalized**] Funding is thereby part of the *grammar and one rewrite* of an ordinary authored-style language — not a side condition bolted onto an evaluator — which is what makes the output of Cost a candidate input to Cost again.

7.4 Morphisms and the ordered category

Continued objects form a category. A morphism is an iGSLT theory map that preserves the ordered cut (core contact, both operand constructors, core pattern, and envelope), reflects the interacting-sort fiber, preserves and reflects the finite wrapped-constructor closure, and commutes with canonicalization injectively on canonical keys. The forgetful functor to iGSLTs forgets object structure but no morphism action. [**formalized**] On top of this sits the *ordered* category: canonical keys carry a collision-free structural code, giving every object a linear order on keys, and ordered morphisms are those whose key action is monotone. Exact typed Cost objects (Section 10) then form an honest full subcategory — written `CostOneObjects` — which is the intended domain of strict Cost_1. [**formalized**]

8 Typed normalization: a refutation and its repair

The July development exposed a second incorrect hypothesis, this time in our own earlier normalization architecture, and the repair reshaped the whole equational layer. We record both, because the counterexample is compiled and the repair is, we believe, the right general discipline for proof-relevant rewriting over generated languages.

The refutation. The earlier normalization laws demanded a global *raw fiber stability*: that every raw equation-equivalence between patterns in a typed fiber restricts to that fiber. For the rho instance this is *false*, and the witness is compact enough to show in full (`CostCanonicalLaws.lean`, theorem `rho_costOpenEquationFiberStable_not`; both endpoint typing verdicts are kernel-decided). The generated Cost language has two *colours* of every rho constructor — a base copy and a wrapped copy — but the raw *parallel collection* representation is deliberately shared between them, while the two process sorts and their unit constructors are distinct. Now take the

mixed-colour term

$\mathsf{PDrop}_{\mathrm{w}}(\,\mathsf{NQuote}_{\mathrm{b}}(\,\{|\,|\}\,)\,)$ — a *wrapped* drop of a *base* quotation of the empty parallel.

This term is well-typed at the wrapped sort: the empty bag is a valid (base) parallel body, so the base quotation is a valid name. The *wrapped* reflective canonicalizer, however, normalizes the empty parallel to *its own* colour's unit, producing

$\mathsf{PDrop}_{\mathrm{w}}(\,\mathsf{NQuote}_{\mathrm{b}}(\,\mathsf{PZero}_{\mathrm{w}}\,)\,)$ — a *wrapped* zero inside a *base* quotation.

The base quote constructor demands a base-sorted process argument, so the canonical endpoint is ill-typed — yet the two terms are related by a single authored reflective generator step at the hole context, so the raw equational relation connects a typed term to an untypeable one (Figure 1). **[counterexample recorded]** A raw equivalence endpoint is simply too weak a datum: the empty bag does not remember *which colour's* unit it is the degenerate case of, exactly as raw syntax cannot remember the binder slice that typing carries.

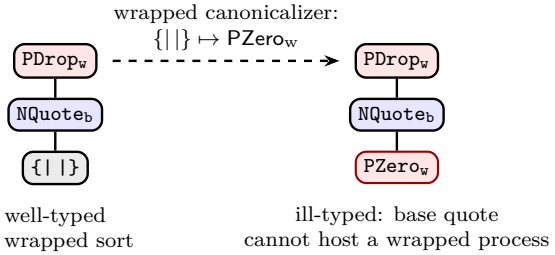


Figure 1: The compiled fiber-stability counterexample. Blue boxes are base-colour constructors, red are wrapped-colour, gray is the colour-shared raw parallel collection. One authored reflective generator step relates the two terms, and the right endpoint is untypeable: the wrapped canonicalizer collapsed the shared empty parallel to its own colour's unit inside a base quotation.

The repair: typed equation carriers. The equational theory is now carried at the typed level. Open patterns in a declaration-derived fiber carry their typing; the authored equations generate a setoid on these *typed* carriers; and the two directions between raw and typed are deliberately asymmetric. Typed-to-raw erasure is free. Raw-to-typed lifting exists only edge-wise: one authored contextual generator step whose *two endpoints are already packaged in the same typed open fiber* may be injected, and the endpoint certificates are the entire distinction from the refuted operation, which attempted to lift an opaque raw path after the fact (`EquationSubstitution.lean`). **[formalized]**

Proof-relevant elaboration. Above the typed carriers sits a proof-relevant elaboration layer: a cost region tree is a typed derivation of how a pattern decomposes into coloured static regions, and normalization is defined on trees, not on raw syntax. Compilation chooses a tree but contributes no semantic equation authority; the executable compact normalizer inherits typed soundness from the elaboration it erases. The normalization theorem lands in the typed open-equation setoid of the exact fiber — with *no* global raw fiber-stability hypothesis anywhere in its statement or proof (`CostRegionNormalization.lean`). **[formalized]**

Exact Cost objects. An object carries *exact* typed canonical laws (`CostOpenCanonicalLaws`) when typed unary normalization is sound in every fiber, compact normalization is coherent across elaboration choices, and each authored open-equation generator is collapsed. On such objects the normalizer is a computable section of every typed open fiber: equivalence classes coincide with canonical representatives, completeness being generated solely from exact invariance under the authored generators. These laws are an object *property* — membership is semantic, not structural — and cut out the full subcategory on which strict Cost_1 lives. [**formalized**]

9 Cost as an endofunctor: the object-closure programme

Iterating Cost means: the generated funded language, with its generated cut and generated retyping plan, is again a continued object. The data triple exists by construction. The closure laws are theorems to be proved *generically* — for every source object — and this programme (`CostEndofunctor.lean`) is now largely discharged:

- every bare-collection constructor of the generated language is non-principal at the next layer (the generated principals never use bare collections, by the source laws) [**formalized**];
- the generated interaction envelope is stable under the next retyping, by transport of the source envelope-stability witness [**formalized**];
- there is a single declaration-derived typing morphism from the complete generated language into the signature that validates the next layer (`costClosureTyping`), reusable by the equation, contractum, and reflective closure proofs [**formalized**];
- the generic typed transport along a retyping plan (`mapCostStaticGenerated`) is stated and proved over the minimal non-circular signature described in Section 7. [**formalized**]

Every remaining object obligation — sorted base and wrapped images of the generated equations, sortedness of the twice-retyped redex and contractum, reflective-presentation survival, canonical preservation of the next non-principal fragment, and the contextual strengthening of the typed canonical section — has since been discharged: one application of Cost is now a fully constructed continued object, every field either derived from the generated declaration or supplied by an explicit object law. [**formalized**]

What remains open is subtler: closure of the exact *laws* themselves. A recorded cross-colour counterexample shows that the compact exact carrier is not preserved as currently generated: at the second layer, the wrapped colour's sort action moves only the interacting sort, so the two colour-owned parallel units become typed in *one* shared fiber, where the colour-blind collapse of the shared empty parallel relates both to one term — one typed equation class then contains two colour-distinct normal forms, and no colour-local normalizer can be invariant on it. (The first layer escapes precisely because its two units are sort-separated — the same mechanism as the counterexample of Section 8.) The repair is a genuine fork, currently under decision: either the proof-relevant elaborated relation becomes the endofunctor's primary carrier, with compact exactness a property of faithful objects only, or the wrapped colour's sort action is made hereditarily injective on *all* sorts, restoring colour-disjoint fibers at every layer. [**open**] Two structural facts shape the endgame either way: closure lands on the exact full subcategory, and the uniform shortcut is provably unavailable — the twice-retyped redex cannot be transported by any single signature morphism, because the declared cut-aware principal copies disagree with the uniform image at exactly the two continuation parameters.

10 Canonical forms and the computable section

Signature keys, and every quotient used above, presuppose a *computable canonical form* for the two static equivalences of the pure calculus. A canonicalizer nf must:

- normalize binders so that alpha-equivalent terms coincide (locally nameless or de Bruijn representation);
- normalize processes and names mutually, with termination measured by quote depth;
- orient the name cancellation $@(*x) \equiv_N x$ as a directed simplification;
- flatten parallel composition, remove 0, normalize components recursively, and sort them under a proved total order on terms;
- keep name equivalence and process congruence as *separate* sorted relations — the canonical form respects both without merging them into one relation over raw syntax.

The obligations are soundness ($\mathsf{nf}(P) \equiv P$), completeness on the pure carrier ($P \equiv Q \Rightarrow \mathsf{nf}(P) = \mathsf{nf}(Q)$), and idempotence. [**formalized**] The proved carrier consists of patterns without the optional finite-set extension. Canonicalization preserves that subtype and descends by quotient lifting to a computable representative function; the representative is proved to belong to its original equivalence class. The continued-cut presentation now stores this explicit representative carrier and embedding instead of selecting representatives with `Quotient.out`.

The mathematical key is the canonical form itself. A runtime digest may later refine it under an explicitly assumed collision-resistance property, and the formalization never treats a concrete hash function as injective.

In the formal development this section is *contextual and open*: it is indexed by every declaration-derived open sorted fiber — free-variable context, binder context, and reflective scope included — and additionally preserves free-variable support, is natural under agreeing recontextualization of unused entries (exact raw representative equality, not merely contextual equivalence), and preserves reflective support. The closed paper-facing section above is its restriction to the closed interacting fiber. The typed Cost normalizer of Section 8 is held to the same interface. [**formalized**] for the pure section; the contextual strengthening of the Cost-side section is part of the open endofunctor obligations of Section 9.

11 One authored presentation and semantic agreement

The pure syntax, equations, reflective constructors, and COMM rule are authored once in the rho language definition. The derived well-sorted judgments supply the induction principles for names, processes, and parallel lists. On that derived process carrier, reflective substitution compiled from the authored declaration is proved exactly equal to the independently developed semantic COMM substitution. Every supported COMM contractum compiled from the authored rule is therefore both a step of the GSLT derived from that same declaration and an agreement point with the established semantic relation. [**formalized**]

The restriction to the derived process judgment is essential. The shared raw pattern type also contains ill-sorted terms, including quotations placed directly in process position; no global agreement theorem is claimed for those terms. During the proof, this restriction exposed and led to the repair of an overlapping unary-pattern dispatcher that could descend beneath a quotation. A regression example now fixes that boundary.

12 Scoped funded execution refines pure rho

The raw de Bruijn representation is intentionally larger than the object language: it can express an index that is outside its enclosing binder, and it can retain such an index underneath a quotation. These values are useful as negative tests, but they are not rho terms. Following the standard scope-safe-syntax discipline [8], the funded syntax has a derived *binder-safe* judgment tailored exactly to pure erasure. It checks all syntax observable after erasure; purse locations, which the pure projection removes, are deliberately outside this judgment. A separate executable runtime-scope predicate checks every raw field, including purse locations, and public admission requires it together with grammar validity. Runtime scope is proved to imply the pure-erasure binder-safe judgment. A quotation seals the surrounding binder, so a term quoted inside a receiver body must already be closed with respect to that body. The judgment is derived over the existing cost syntax; it neither adds a second syntax nor changes COMM.

The refinement result has the usual initialization–preservation–simulation shape. First, flattening a binder-safe source term into a funded configuration preserves binder safety. Second, every declarative funded step maps a binder-safe configuration to a binder-safe configuration. Third, cost substitution on this fragment agrees, up to the *pure* `StructuralCongruence`, with the established semantic COMM substitution. The proof uses the pure canonical section and never an equivalence containing the executable free-Drop rule. [**formalized**]

Configuration erasure is defined by quotient lifting over permutation of funded components, never by selecting a representative with `Quotient.out`. Each binder-safe `CostStep` then erases to one step of the pure GSLT derived from `rhoCalc`. Induction over declarative paths gives a rewrite path in that same derived GSLT; its length is exactly the number of funded events and exactly the length of the path's ordered demand receipt. Thus erasure inserts neither administrative reductions nor executable-Drop reductions. [**formalized**]

The executable/declarative commuting edge is also formalized. Every publicly admitted budgeted Lean execution yields its occurrence-bearing runtime path, a same-length declarative funded trace, and a rewrite path of that same length in the pure GSLT derived from `rhoCalc`. Structural normalization seams remain explicit; they are not replaced by syntactic equality. [**formalized**]

At the parallel boundary, every list of enabled modeled runtime candidates whose complete endpoint-and-funding source-index family is globally duplicate-free lifts to a selected runtime wave. The wave's witnessed source partition constructs a fixed compatible cost matching, and every permutation of that matching serializes as an ordinary declarative cost trace. This is a theorem about the Lean runtime model under its occurrence-claim postcondition; it is not a universal theorem that the compiled C implementation satisfies the postcondition. [**formalized**]

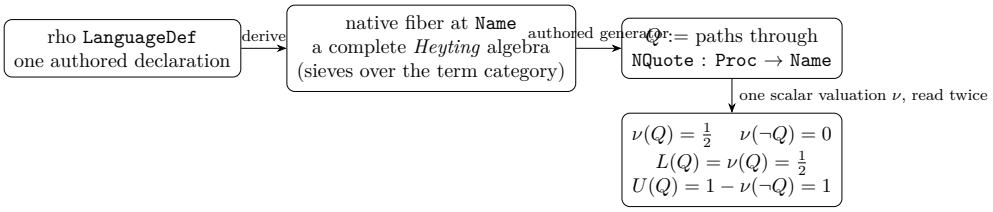
The compiled C engine is checked independently by differential execution and Lean receipt replay, including rejection of tampered receipts. That evidence is deliberately not presented as a universal theorem about compiled C.

13 The native observation logic: two bounds are forced

Everything so far graded *steps*: cost decorates the rewrite relation and accumulates along runs. This section concerns the other axis — what a *formula* returns when evaluated against the calculus — and proves that for rho's own native observation logic the answer cannot be a single number.

The native type theory derived from the authored presentation assigns to each sort a fiber of predicates: sieves in the presheaf category over the term syntax. These fibers are complete Heyting algebras, and for rho the interesting one sits at `Name`, because rho has an authored constructor into

it:



The predicate Q is generated by `NQuote`: it contains exactly the paths into `Name` that factor through quoting. Naturality then pins both sample points: `NQuote` itself lies in Q, the identity on `Name` does not, and the complement cannot contain the identity while excluding its restriction along `NQuote`. Three facts follow, each kernel-checked:

1. **The fiber is not Boolean.** $Q \vee \neg Q \neq \top$: excluded middle fails at an authored predicate, not an imported abstract example.

2. **One ruler, two readings.** The valuation ν is a single scalar map on the fiber. Reading it directly gives the *support* $L(Q) = \nu(Q)$; reading it through negation gives the *non-refutation* $U(Q) = 1 - \nu(\neg Q)$. In a Boolean algebra excluded middle forces $L = U$; here they differ: $L(Q) = \frac{1}{2}$, $U(Q) = 1$.

3. **No point value serves both roles.** Any single number assigned to Q either understates what is not refuted or overstates what is supported. A complete plausibility report about Q is the pair (L, U).

Two scope notes keep the claim exact. First, the witness is *structural*: it arises from the constructor/presheaf geometry of quoting, not from COMM nondeterminism, and connecting the same gap to operational $\Diamond/\Box$ observation is future work. Second, the theorem concerns two independently meaningful readouts, not encoding cardinality — a pairing function could pack (L, U) into one real; what it cannot do is make one *semantically interpreted* point value play both roles. Relatedly, the representation-theoretic reading is one-way in exactly the sense of Knuth–Skilling fidelity: the lattice order constrains admissible scalarizations, and their two-component quantum result requires a further pair postulate; neither is claimed here.

The consequence for the value dial of an observation discipline over rho: one may still choose *which* two-component algebra to reason in — evidence pairs and amplitude–phase are algebraically different choices with different capabilities — but the choice between point-valued and interval-valued readouts is not free. Rho's own native logic already contains a predicate that makes the point-valued option insufficient.

14 Provenance and status

The formalization pins the exact manuscript versions it implements — title, date, and content digest recorded alongside the theory — because a silently updated source would change the meaning of proved names. The operative sources are the reflective higher-order calculus paper of Meredith and Radestock; the graph-enriched Lawvere-theory account of operational semantics of Stay and Meredith; and the June 2026 versions of the continued-interactive-GSLT/cost and cost-accounted-rho manuscripts, whose earlier circulated revisions differ materially and are kept only as comparison history.

Item	Section	Status
COMM-only concrete one-step relation	§2	formalized
Free drop inert in pure rho (negative theorem)	§2, §5	formalized
Runner idiom (one-COMM execution)	§3	stated; example-level
Engine: located funding, receipts, budgets, waves	§4	implemented; gates green
Engine/Lean boundary: prefix agreement, receipt replay	§4	differential evidence
Execution delta with positive/negative theorems	§5	formalized
Fixed-element cancellation boundary	§6(a)	formalized
μ joint-injectivity impossibility	§6(b)	formalized
Parameterized monad laws on funded execution	§6	formalized
Exact receipt / lossy aggregate factorization	§6	formalized
Continued category (objects, morphisms, ordered keys)	§7	formalized
Generated funded language: whole-redex rule, positive shape	§7	formalized
Generic typed transport over (theory, cut, plan)	§7	formalized
Raw fiber stability for rho	§8	counterexample recorded
Typed equation carriers; edge-wise raw-to-typed lifting	§8	formalized
Proof-relevant elaboration; typed normalization soundness	§8	formalized
Exact canonical laws; `CostOneObjects` subcategory	§8	formalized
Endofunctor: one Cost application as a continued object	§9	formalized
Endofunctor: closure of the exact laws themselves	§9	open; cross-colour counterexample recorded
Computable pure canonical form + section	§10	formalized
Authored COMM compiler agreement with established semantics	§11	formalized
Binder-safe initialization and preservation	§12	formalized
Funded-step erasure to the LanguageDef-derived pure GSLT	§12	formalized
Runtime and declarative paths to derived pure paths, exact length	§12	formalized
Occurrence-disjoint modeled runtime waves to fixed cost matchings	§12	formalized
Pure/extended and cost-profile execution boundary	§5	Lean theorems; C differential gate
Native fiber non-Boolean; $L < U$ forced at authored predicate	§13	formalized

References

[1] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(3):49–67, 2005.

[2] M. Stay and L. G. Meredith. Representing operational semantics with enriched Lawvere theories. 2017.

[3] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor: wrapping by construction; and Cost accounting for the reflective higher-order calculus. Manuscripts, June 2026 versions (pinned by digest in the repository).

[4] F1R3FLY-io. *mettail*: a reference implementation family for rho-style calculi. Open-source repository; used here as a bounded encoding reference, with its implementation delta recorded in Section 5.

[5] R. Atkey. Parameterised notions of computation. *Journal of Functional Programming*, 19(3–4):335–376, 2009. `https://bentnib.org/param-notions.html`

[6] S. Katsumata. Parametric effect monads and semantics of effect systems. In *Proceedings of POPL*, 2014. `https://group-mmm.org/~s-katsumata/paper/ppl2014.pdf`

[7] D. Orchard, P. Wadler, and H. Eades III. Unifying graded and parameterised monads. In *Proceedings of MSFP*, 2020. `https://arxiv.org/abs/2001.10274`

[8] G. Allais, R. Atkey, J. Chapman, C. McBride, and J. McKinna. A type- and scope-safe universe of syntaxes with binding: their semantics and proofs. 2021. `https://arxiv.org/abs/2001.11001`